PRODUCT REQUIREMENTS DOCUMENT

Paper2Run
产品需求文档

论文自动复现引擎：把论文，变成能跑的实验

参赛赛道：赛道一 效率进化

特别赛题：基于 Qwen 大模型的科研提效（阿里云）

文档版本：V1.0

发布日期：2026 年 9 月

Paper2Run 项目组　　天猫 AI 黑客松

目　录

1 产品概述 1

1.1 一句话定位 1

1.2 电梯陈述 1

2 目标用户与使用场景 1

3 痛点与价值主张 2

3.1 四个核心痛点 2

3.2 价值主张 2

3.3 产品三原则 2

4 功能规划 2

4.1 MVP（P0）功能 2

4.2 后续版本功能 3

5 核心用户旅程 4

6 页面结构与原型说明 4

6.1 全局框架 5

6.2 工作台（入口） 5

6.3 论文解析（人在决策点） 5

6.4 生成与执行（Agent 工作台） 6

6.5 复现报告 6

7 产品原理与技术架构 6

7.1 五层架构 6

7.2 修复循环（核心技术） 7

7.3 复现评分设计 7

8 关键指标 7

9 风险与对策 8

10 与 Agent 协作的建造路线 8

附录　命名与口号 9

注：本目录由域代码生成。文档编辑后如需刷新页码，请在目录上点击右键并选择「更新域」。

1 产品概述

1.1 一句话定位

Paper2Run 是一台论文复现机：输入一篇 arXiv 论文 PDF，输出一套已经跑通的可执行实验，以及一份与原文逐项对照的复现报告。它瞄准的不是想出研究想法的环节，而是把想法变成可验证事实时最耗时的一步：复现。

1.2 电梯陈述

科研里最耗时的环节往往不是想 idea，而是复现：论文代码缺失、环境跑不起来、结果对不上，研究生常常要花两周才能拿到一个基线数字。Paper2Run 把这段工作交给 Agent。上传 PDF 之后，系统用 Qwen-VL 阅读论文、Qwen3-Max 负责规划 Qwen3-Coder 编写代码，并在隔离的 Docker 沙箱中自动运行；一旦跑不通，报错日志会回喂给 Agent 自动修复，循环直到跑通为止。最终输出指标对比表、曲线对照图，以及一份可核查的复现声明。整个过程中人只需要点三次按钮：上传、确认解析、查看报告。

2 目标用户与使用场景

表 1 目标用户与场景

用户	场景	现状耗时	使用 Paper2Run
研究生、博士生（主力）	读到一篇想借鉴方法的论文，需要复现基线	2 天至 2 周	上传后等待 30 分钟至数小时，得到可运行代码与对比报告
算法工程师	快速验证论文方法是否值得引入业务	1 至 3 天	同上，且实验环境一键打包带走
论文评审与导师	判断论文结果是否可信	只能看图，无法执行	直接查看复现得分与偏差表

同样重要的是明确不服务的用户：需要大规模训练的复现（例如大模型预训练）与非机器学习领域的论文，在 V1 中明确排除，以保证核心闭环的稳定。

3 痛点与价值主张

3.1 四个核心痛点

1. 论文不放代码，或者放出的代码与文章对不上。
2. 放了代码但环境跑不起来，依赖与 CUDA 版本地狱消耗大量时间。
3. 复现结果与论文偏差大，无法判断是自己错了还是论文错了。
4. 读懂方法再翻译成自己框架的代码，这段人工转换重复且低价值。

3.2 价值主张

从读过到跑过：以执行结果为准绳的复现闭环，而不是聊天式的猜测。代码跑没跑通、指标差多少，由沙箱给出答案。

报错即燃料：执行失败不是终点，而是下一轮修复的输入。错误日志、Agent 诊断与补丁构成一个自动收敛的循环。

可核查的诚实：得分、偏差、随机种子、环境锁定全部披露。复现不成功时，也明确说清差在哪里、差多少。

3.3 产品三原则

以下三条原则贯穿原型与后续开发中的每个决策，也是答辩时解释设计取舍的依据。

1. 可验证性优先：任何结论必须能追溯到日志与图表，不允许无法回查的断言。
2. 人留在关键决策点：解析确认与最终报告两处必须有人参与，其余环节自动完成。
3. 失败是产品的一部分：复现失败时输出诊断报告，而不是一堆无人能读的报错。

4 功能规划

4.1 MVP（P0）功能

MVP 指黑客松现场演示必须闭环的八项功能，缺一不可。

表 2 MVP 功能清单

编号	功能	说明
F1	论文上传	拖拽 PDF 或粘贴 arXiv 链接
F2	论文解析	提取任务类型、数据集、指标、方法实体、关键公式与图表清单，输出结构化 JSON
F3	解析确认页	用户核对并修正提取结果，是人留在决策点的第一处
F4	代码生成	Qwen-Coder 按解析结果生成数据加载、模型、训练、评估与依赖清单
F5	沙箱执行	Docker 容器内运行，限时限量，日志实时回流前端
F6	修复循环	报错回喂、Agent 诊断、打补丁、重跑，上限 5 轮，时间线可回放
F7	复现报告	指标对比表、曲线对比图、复现得分与偏差诊断
F8	资产下载	代码包、Docker 镜像、训练日志与报告 PDF

4.2 后续版本功能

表 3 V1 与 V2 功能规划

版本	功能	目的
V1	复现项目历史与版本对比	同一论文多次复现，观察差异
V1	小样本模式	用数据集 5% 子集先快速验证流程，再决定是否全量
V1	循环中人工介入	允许用户在修复循环中插话，例如建议调小学习率
V2	批量复现	一个课题组的全部基线一次跑完

版本	功能	目的
V2	复现社区	公开复现报告，积累论文可信度数据
V2	消融助手	自动复现论文中的消融实验表

5 核心用户旅程

以复现 ResNet（Deep Residual Learning for Image Recognition）为例，完整旅程共八步：

1. 用户上传 ResNet 论文 PDF。
2. 系统解析出图像分类、CIFAR-10、Top-1 指标，以及残差块、批归一化、SGD 等方法实体。
3. 用户确认解析结果，可现场修改，例如把数据集从 ImageNet 换成 CIFAR-10 以缩短演示时间。
4. Agent 生成 train.py、model.py、dataset.py 与依赖清单。
5. 沙箱第 1 轮运行失败：损失变为 NaN。
6. Agent 诊断为学习率过大，打补丁后第 2 轮通过并开始训练。
7. 训练完成，指标提取：Top-1 为 93.72%，论文报告 94.10%，偏差负 0.38 个百分点。
8. 生成复现报告，得分 87 分，判定核心结论成立，一键打包下载。

黑客松演示脚本即这八步：现场在第三步修改一个参数，展示人在决策点的价值；回放第五、六步的修复时间线，展示执行反馈闭环的运转。

6 页面结构与原型说明

可点击的静态原型见文件 Paper2Run-原型.html，四个视图通过左侧导航切换，页面内所有数据为演示用途。整体采用工程蓝图视觉语言：深蓝底色与制图线框呼应「把论文变成机器」的产品本质；衬线字体只用于来自论文原文的内容（论文标题、复现得分、纸片化的 PDF 页面），等宽字体用于机器产出的内容（代码、日志、指标）；米色纸片仅出现在论文原件上，是全站唯一的暖色材料。

6.1 全局框架

表 4 页面全局结构

区域	内容
顶栏	产品名 Paper2Run、模型选择器（Qwen3-Coder 与 Qwen-VL）、运行状态
装置条	六道工序横向贯穿页面：上传论文、解析提取、生成代码、沙箱执行、修复循环、复现报告。状态随当前视图变化，完成显示对勾，进行中高亮呼吸，未开始保持描边。这是全站的签名元素，因为产品本质就是这条流水线
侧栏	四个视图导航与当前项目卡片（论文标题与循环状态）
主内容区	工作台、论文解析、生成与执行、复现报告四个视图

6.2 工作台（入口）

顶部一句话价值主张「把论文，变成能跑的实验。」，下方是上传区：支持拖拽 PDF、粘贴 arXiv 链接，并提供三篇示例论文快捷入口（ResNet、Transformer、DQN）。再往下是最近项目表，刻意展示四种状态：已完成（附复现得分）、修复循环中、失败转诊断（U-Net 超过循环上限的例子）、排队中，让评审直观理解失败也是产品的一部分。

6.3 论文解析（人在决策点）

左栏是米色纸片化的 PDF 原文预览，保留论文的标题、图表与公式质感；右栏是提取结果，分四组卡片：任务画像（数据集等字段可编辑）、方法实体芯片、关键公式、复现对比目标（可勾选论文中的表格与曲线）。右上角显示解析置信度。底部是确认按钮，点击后进入 Agent 工作台。这一页是「人留在关键决策点」原则的物理体现：解析错了，人在这里兜底。

6.4 生成与执行（Agent 工作台）

演示的核心画面，三栏结构：左侧文件树展示生成的项目文件与各自状态；中间是带行号与语法高亮的代码视图，代码注释中标注了修复循环修改过的行；右侧上半是沙箱执行日志的终端面板，红绿琥珀三色区分错误、成功与 Agent 动作，下半是修复循环时间线，每一轮显示错误、Agent 诊断与补丁内容。底部状态栏以分隔线呈现沙箱环境、运行时长、自动修复次数与预计成本。

6.5 复现报告

顶部是复现得分（大号衬线数字）与三项构成：指标吻合度、流程完整度、环境可复刻度，右侧给出核心结论是否成立的判定。左侧是指标对比表（论文报告值、复现实测值、偏差）与曲线对比：论文原图以纸片呈现，复现曲线在深色面板中与论文参考曲线叠加对照。右侧是复现声明（随机种子、代码版本、依赖锁定、循环轮次全部披露）与四项资产下载，支持公开分享复现报告。

7 产品原理与技术架构

7.1 五层架构

表 5 技术架构分层

层次	职责	技术选型
解析层	抽取 PDF 文本与图表，结构化为任务画像 JSON	PyMuPDF、Qwen-VL
智能层	阅读图表、规划任务、编写与修复代码	Qwen3-Max、Qwen-VL、Qwen3-Coder
执行层	隔离运行生成代码，限时限量，日志回流	Docker
编排层	任务状态机与事件流，推送到前端	FastAPI、SSE
展示层	四视图工作台	React

7.2 修复循环（核心技术）

生成的代码进入沙箱执行。执行失败时，错误日志连同当前代码一起回喂给 Agent，由它诊断原因、输出补丁、重跑验证，循环上限 5 轮。每一轮的错误、诊断差异与结果四元组全部落库，前端时间线可逐轮回放。

这套机制成立的三个理由，也是答辩时的话术：

1. 执行反馈是比人工审查强得多的信号：代码跑没跑通、损失是否为 NaN，机器自己知道，不需要人判断。
2. 编写代码加自动调试恰好落在当前大模型最强的能力区间，模型既有代码知识又有工具调用能力。
3. 失败可解释：保留完整的循环记录，最终报告能说清复现究竟难在哪里。

7.3 复现评分设计

复现得分由三部分加权构成：指标吻合度占 60%，流程完整度占 25%，环境可复刻度占 15%。评分的同时强制披露随机种子、依赖锁定、代码版本与每轮循环日志。设计取向是宁可诚实扣分，不做虚假通过，这是与模型幻觉对抗的产品设计。

8 关键指标

表 6 北极星与过程指标

指标	类型	目标
复现闭环成功率（上传到产出有效报告）	北极星	不低于 60%（限定 CV 分类论文范围）
平均修复循环次数	过程	不超过 3 轮
端到端耗时	过程	不超过 1 小时
虚假通过率（报告与实际不符）	护栏	接近 0，靠声明与日志兜底

9 风险与对策

表 7 主要风险与应对

风险	对策
大论文、大训练规模跑不动	演示限定小数据集与少轮数；V1 提供子集采样模式
Agent 修复陷入死循环	循环上限 5 轮，超限自动转为诊断报告输出
解析错误传导到全链路	解析确认页由人工核对兜底
沙箱安全	无网络容器、资源限额、产物只出不进

10 与 Agent 协作的建造路线

本产品由人提出需求、Agent 执行建造。协作纪律是：每个阶段只给 Agent 一个明确目标与一条验收标准，通过后才进入下一阶段；每阶段结束时让 Agent 写一段阶段总结存档，作为下一阶段的上下文。人只做验收员。

表 8 分阶段建造路线

阶段	目标	验收标准
0 定稿	PRD 与原型定稿，Agent 复述对产品的理解	正确列出 MVP 功能清单（已完成）
1 骨架	按原型搭建前端，四个视图用假数据渲染	浏览器中四个视图可切换，观感与原型一致
2 解析层	PDF 解析为结构化 JSON 并展示在确认页	上传 ResNet 论文后字段正确
3 生成层	结构化 JSON 组装为项目代码	生成代码结构完整并引用解析出的数据集
4 执行层	Docker 沙箱运行与日志实时回流	上传含缺陷的代码能看到报错流出
5 修复循环	错误回喂、诊断、补丁、重跑，时间线可回放	故意设置过大学习率，观察被自动修复
6 报告层	指标提取、偏差对比、得分与打包下载	端到端跑完第 5 章的八步旅程

阶段	目标	验收标准
演示准备	两篇预验证论文与兜底录屏	现场台词与交互节奏熟练

给 Agent 的第一句话建议为：先读 PRD 与原型文件，复述你对产品的理解并列出 MVP 清单，不要开始写代码。阶段四是技术难点，最容易翻车，应预留最多时间。

附录　命名与口号

产品名 Paper2Run，含义是论文到跑起来。中文口号：把论文，变成能跑的实验。答辩收尾一句话：别人问 AI 能不能加速科研，我们让 AI 先干科研里最脏的活，复现。